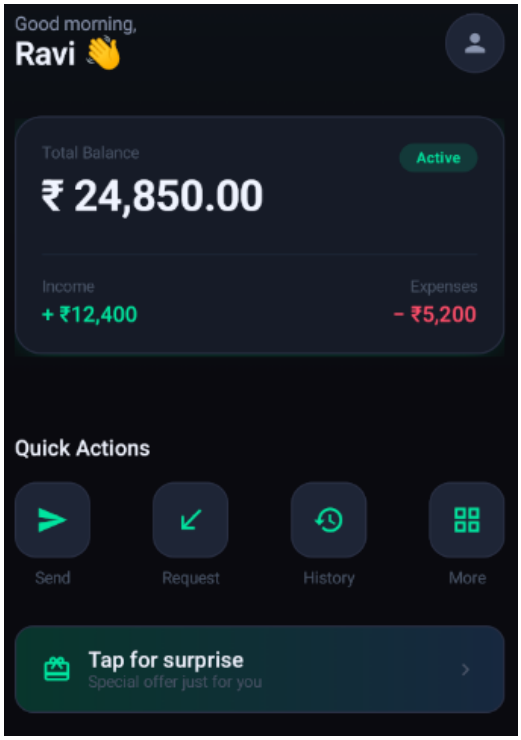
Full Walkthrough of All 8 Flags

22PC12 - G Karthik

22PC18 - Mahadev Manohar

Flag: `Payment{init_easy}`

- **Technique:** `adb logcat` leak
- **Walkthrough:**
    1. Run the application on an emulator or physical device.
    2. Start monitoring the system log by running the `adb logcat` command in your terminal.
    3. In the application, click the 'tap for surprise' button.



4. The flag will appear directly in the `adb logcat` output as a leak.

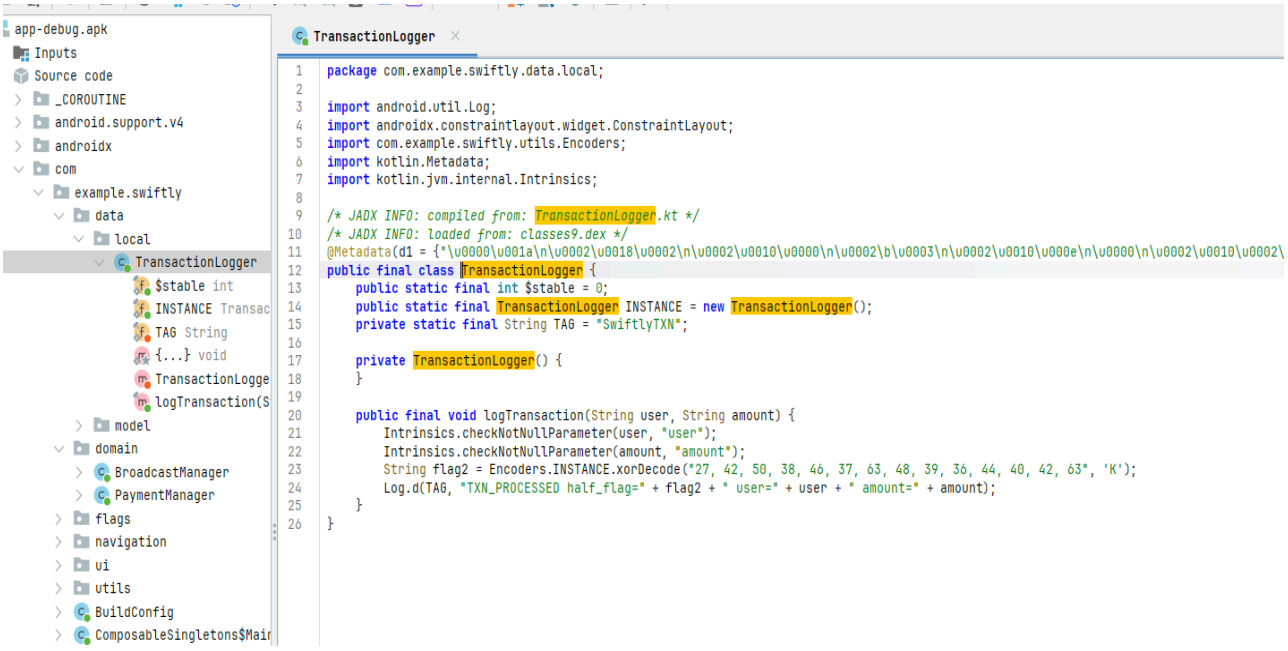
```
MobileSecurityLab]$ adb logcat | grep "Payment{"
7.321  5131  5131 D SwiftlyApp: init_check: Payment{init_easy}
```

First Flag => `Payment{init_easy}`.

Flag: `Payment{logcat_flag_2}`

- **Technique:** `adb logcat + XOR decode`
- **Walkthrough:**
    1. Trigger a payment process within the application.

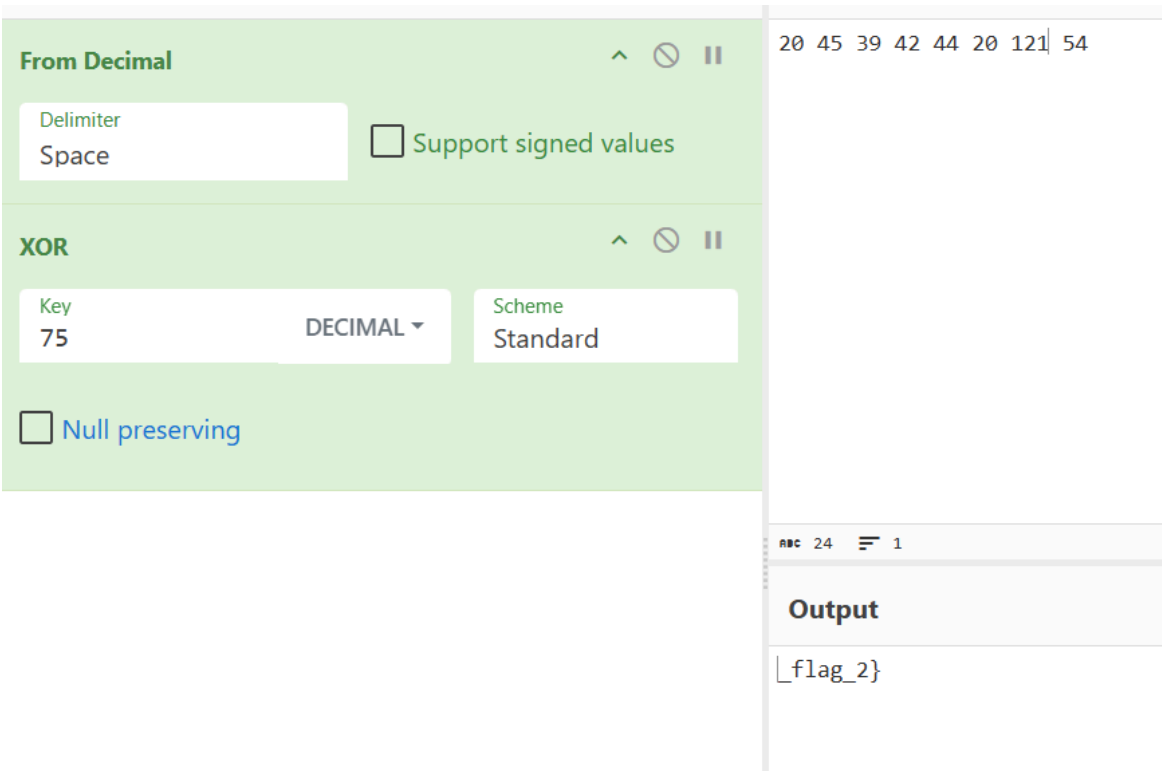
2. Capture the resulting log message using `adb logcat`, which will contain the flag obfuscated by a simple bitwise cipher.
3. Use the identified XOR key and a decoding tool or script to reveal the plain-text flag.
4. The second half of the flag is located in the StringUtils kotlin file as 3 parts of object which is to be decoded with the same key for the first half



String flag2 = Encoders.INSTANCE.xorDecode("27, 42, 50, 38, 46, 37, 63, 48, 39, 36, 44, 40, 42, 63", 'K');

On decoding this, u get the value as **Payment{logcat**

```java
public final class StringUtils {
    public static final int $stable = 0;
    public static final StringUtils INSTANCE = new StringUtils();
    private static final String partA = "20, 45, 39";
    private static final String partB = "42, 44, 20";
    private static final String partC = "121, 54";
```



On decoding this, u get the value as **_flag_2}**

Flag: `Payment{intent_flag_3}`

- **Technique:** `jadx static or adb shell`

- **Walkthrough (using static analysis):**

    1. Open the `app-debug.apk` in `jadx-gui`.
    2. Perform static analysis to find where a broadcast intent is being sent.
    3. Identify the encrypted string contained within the extra data of an unregistered broadcast manager kotlin file

```
public final void sendPaymentBroadcast(Context context, String user, String amount) {
    Intrinsics.checkNotNullParameter(context, "context");
    Intrinsics.checkNotNullParameter(user, "user");
    Intrinsics.checkNotNullParameter(amount, "amount");
    Intent intent = new Intent("com.swiftly.internal.PAYMENT_COMPLETE");
    intent.putExtra("receipt_token", "Pnbdsvw{iawvbb_ilnj_3}");
    intent.putExtra("user", user);
    intent.putExtra("amount", amount);
    context.sendBroadcast(intent);
    Log.i(TAG, "Broadcast dispatched for user=" + user);
}
```

    4. Decrypt the flag using vigenere cipher with key being 'ANDROID'

Pnbdsvw{iawvbb_ilnj_3} => Vigenere cipher with key 'ANDROID' =>

Payment{intent_flag_3}

Flag 4: Persistent Key-Value

- **Flag:** `Payment{shared_prefs_flag_4}`
- **Technique:** `shared preferences xml file or jadx`
- **Walkthrough:**

1. The flag is written to the app's private storage (SharedPrefs) on first launch, in the `swiftly_session` file from HomeScreen UI.
2. Use jadx gui to find the home screen kotlin file to

```
@Override // kotlin.coroutines.jvm.internal.BaseContinuationImpl
public final Object invokeSuspend(Object obj) {
    IntrinsicsKt.getCOROUTINE_SUSPENDED();
    switch (this.label) {
        case 0:
            ResultKt.throwOnFailure(obj);
            SharedPreferences prefs = this.$context.getSharedPreferences("swiftly_session", 0);
            if (!prefs.contains("session_token")) {
                Intrinsics.checkNotNull(prefs);
                SharedPreferences.Editor editorEdit = prefs.edit();
                editorEdit.putString("session_token", "Hwgrxyr{kdiwxo_njanx_ywyy_4}").putString("user_id", "usr_8f3a2c").putString("key4", "app name").
                editorEdit.apply();
            } else {
```

putString("session_token", "Hwgrxyr{kdiwxo_njanx_ywyy_4}").

putString("user_id", "usr_8f3a2c").

putString("key4", "app name")

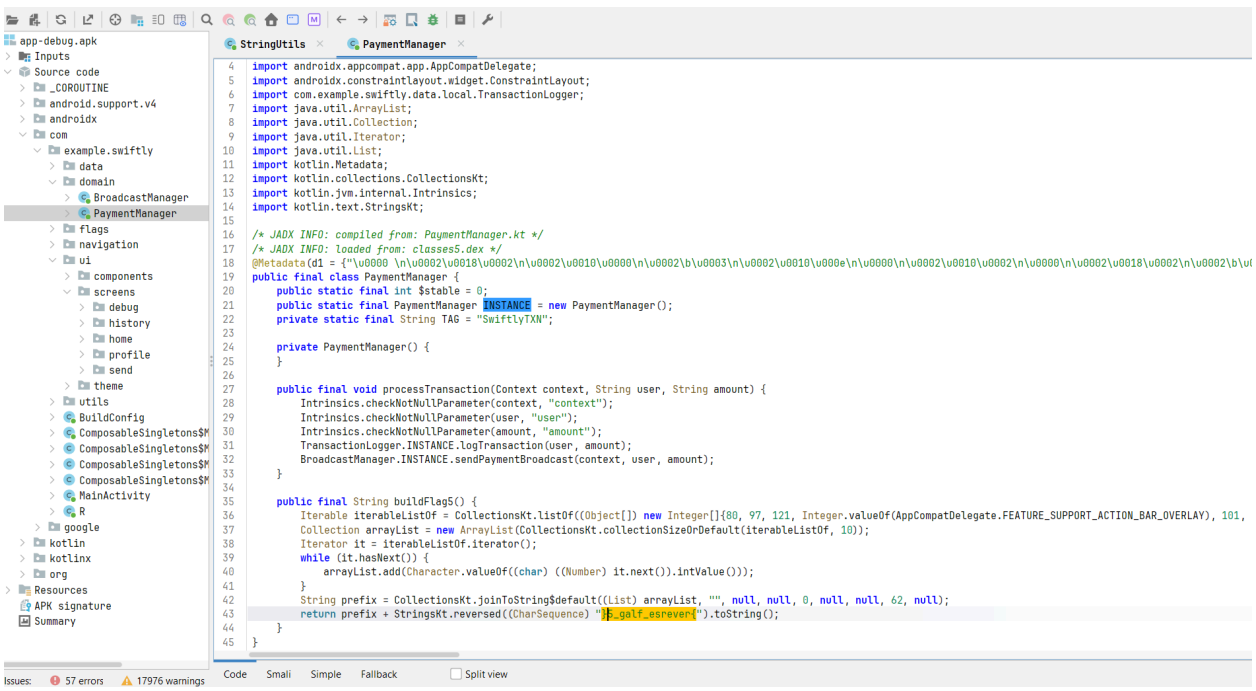
Here key is 'SWIFTLY' and it is the same vigenere cipher as flag 3 but here key is 'SWIFTLY'

After Decrypting => Payment{shared_prefs_flag_4}

Flag 5: Logic Reversal

- **Flag:** `Payment{reverse_flag_5}`
- **Technique:** Static code analysis / string reversal
- **Walkthrough:**
    1. Open `jadx-gui.exe` and load the APK.
    2. Navigate to `com.example.swiftly` → `PaymentManager.kt`.
    3. Locate the function `fun buildFlag5()`.

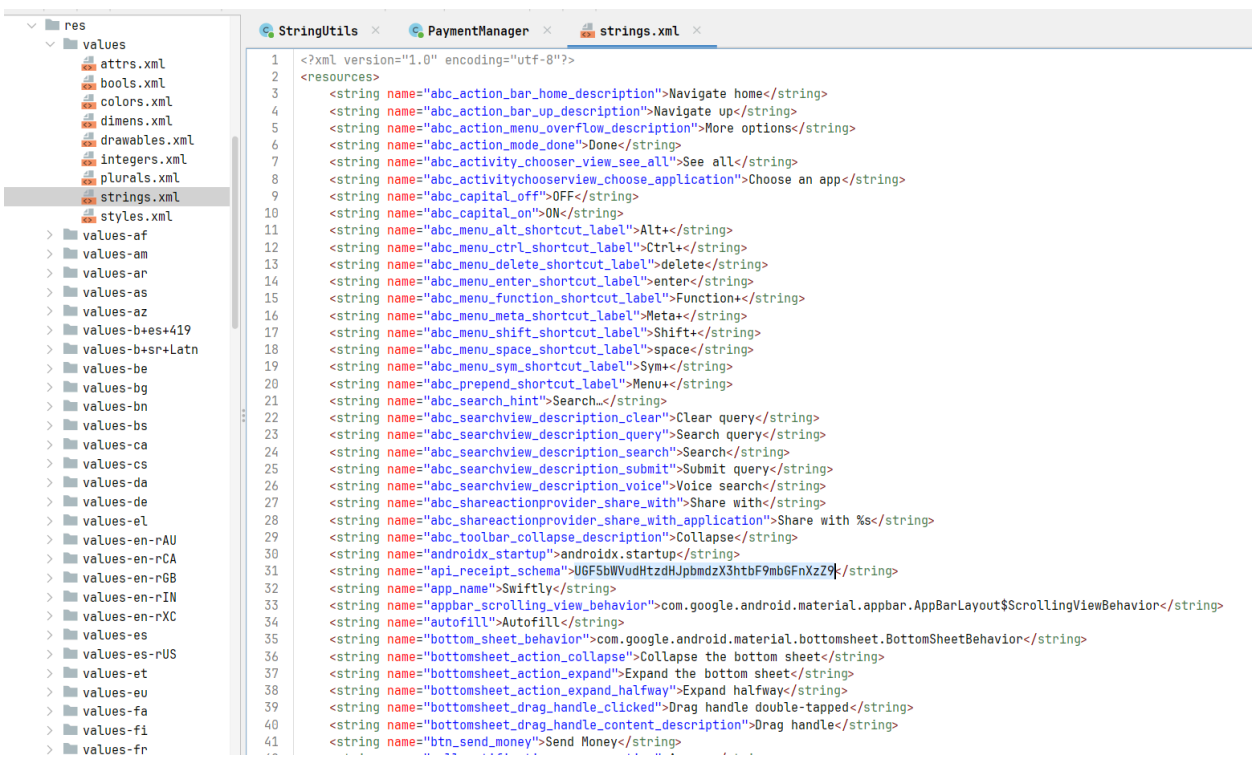
4. Observe that the suffix for the flag is assigned the reversed string: `suffix = "}5_galf_esrever{"`.
5. Reverse the suffix (e.g., using Python: `print("}5_galf_esrever{"[::-1])`) and combine it with the prefix to get the final flag.
6. The trigger in the app is tapping the "debug info" link under the amount field on the Send screen.

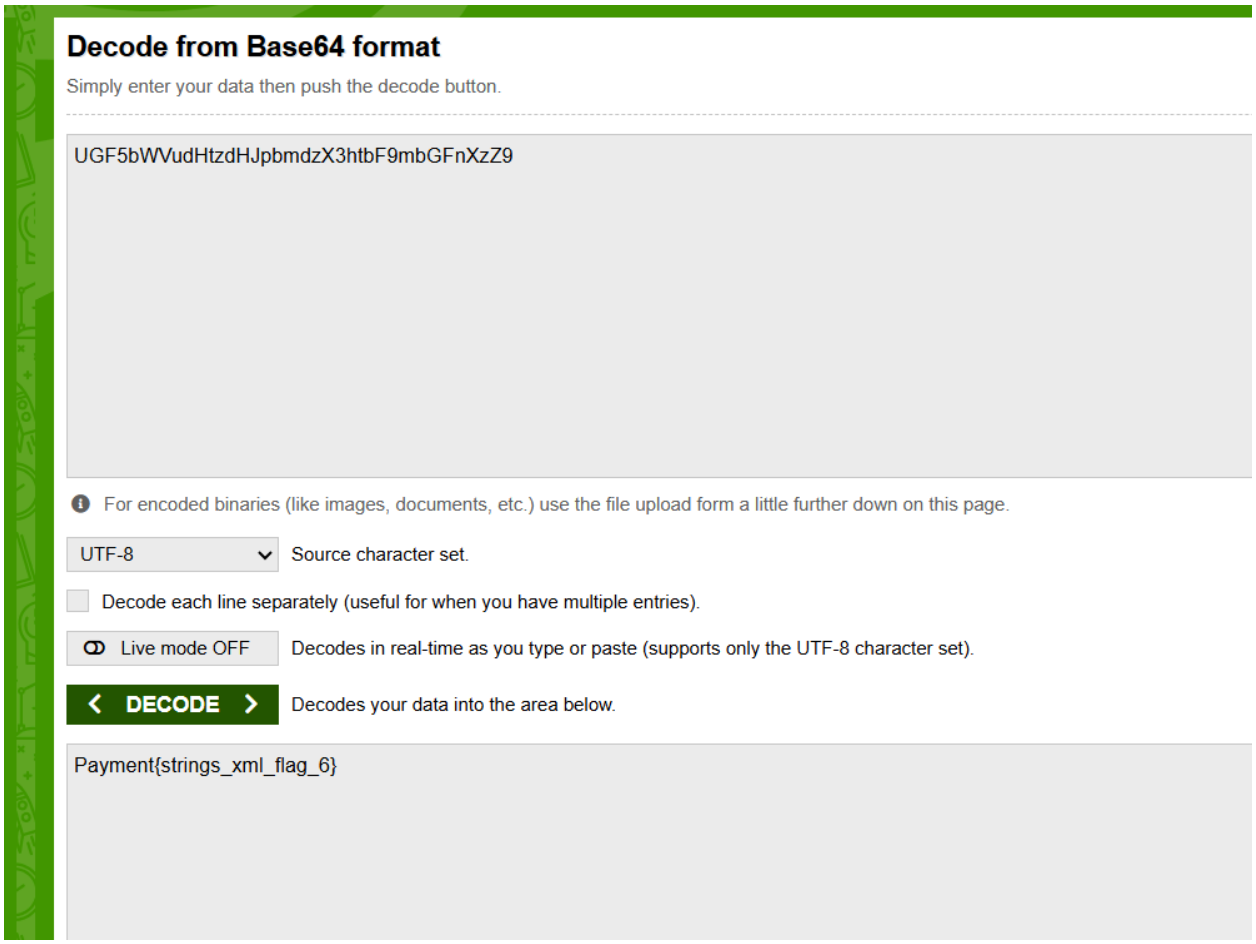


Flag 6: Resource Hidden

- **Flag:** `Payment{strings_xml_flag_6}`
- **Technique:** `jadx resource analysis` or `apktool`
- **Walkthrough (using jadx):**

1. Open the APK in `jadx-gui`.
2. From the top panel, select **Resources** → `resources.arsc` → **res/values/strings.xml**.
3. Find the string with the name `api_receipt_schema`:
   - `<string name="api_receipt_schema">UGF5bWVudHtzdHJpbmdzX3htbF9mbGFnXzZ9</string>`.
4. Decode the Base64 value `UGF5bWVudHtzdHJpbmdzX3htbF9mbGFnXzZ9` (e.g., using `echo "..." | base64 -d`) to get the flag.

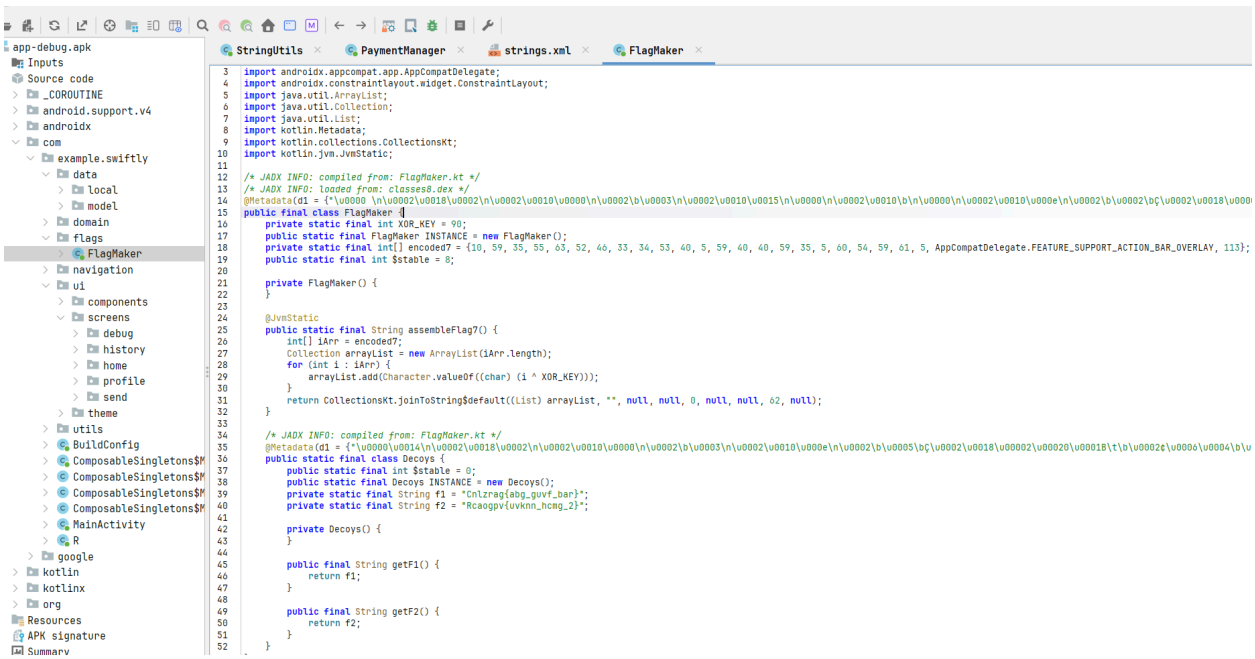




Flag 7: Corrupted XOR Array (FlagMaker)

- **Flag:** `Payment{xor_array_flag_7}`
- **Technique:** Static XOR decode or `Frida dynamic hook`
- **Walkthrough:**
    1. Navigate to `FlagMaker.kt` in `jadx`.
    2. Analyze the `encoded7` integer array and the `assembleFlag7()` function, which uses the key `0x5A` ('Z') for XOR decoding.

3. The main challenge is that one byte in the array is intentionally corrupted to prevent straightforward static decoding.
4. To retrieve the correct flag, you must either identify and correct the corrupted byte manually, or use a dynamic approach like a `Frida hook` on `assembleFlag7()` to capture the flag after it is correctly assembled at runtime.



Using frida:

// frida -U -f com.example.swiftly -l hook_flagmaker.js

Java.perform(function () {

var FlagMaker = Java.use("com.example.swiftly.flags.FlagMaker");

FlagMaker.assembleFlag7.implementation = function () {

```
        var result = this.assembleFlag7();
        console.log("[+] FLAG 7: " + result);
        return result;
    };
});
```

Flag 8: Hidden Debug Screen (Security Bypass)

- **Flag:** `Payment{debug_screen_flag_8}`
- **Technique:** `ADB deep-link + Frida bypass`
- **Walkthrough:**
    1. The flag is in the `flag8Raw` field on a hidden debug screen (`DebugScreen.kt`) that is not linked from the main navigation.
    2. Use an `ADB deep-link` command to navigate directly to the secret route: `debug_hidden`.

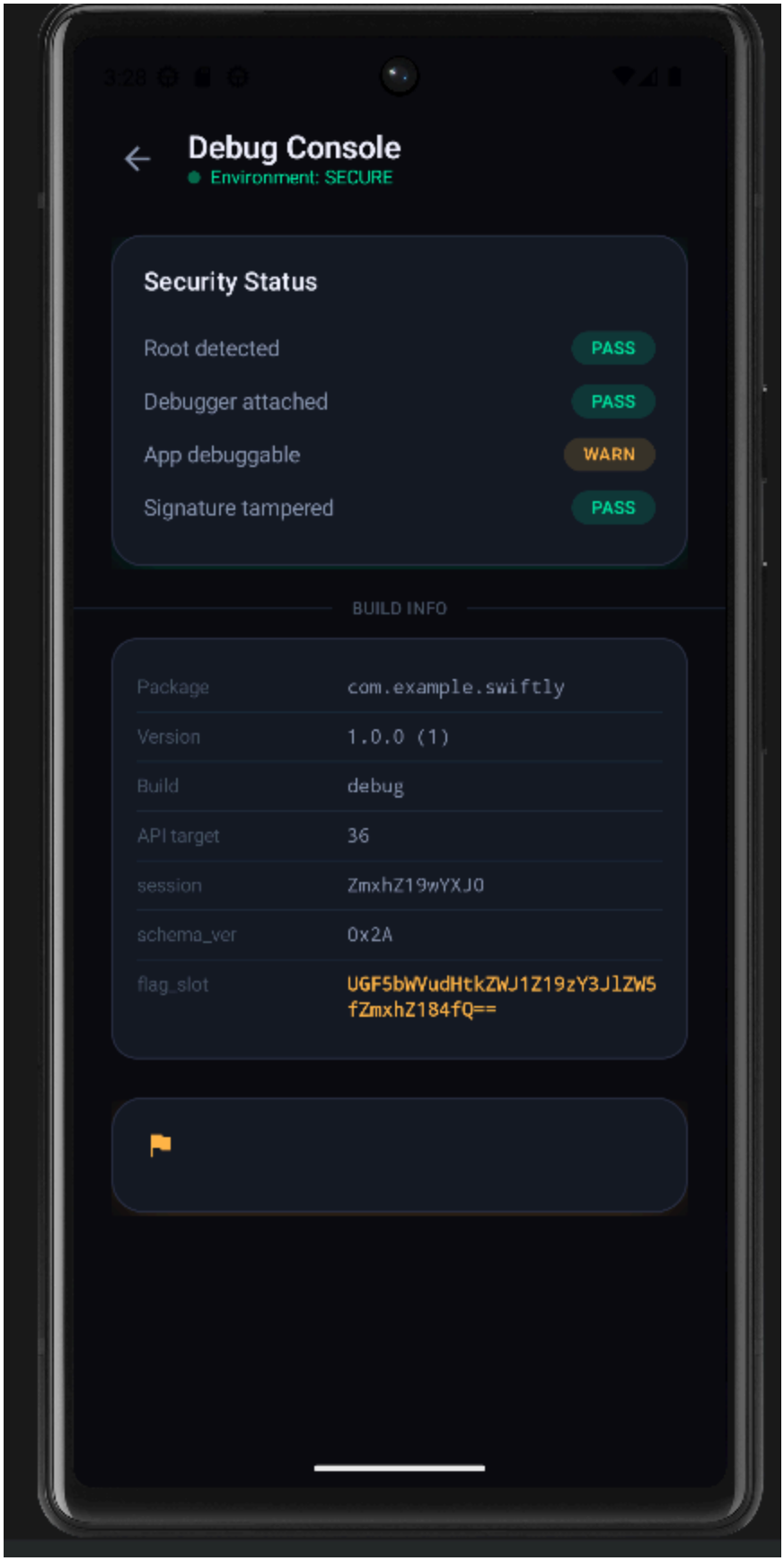
adb shell am start -W -a android.intent.action.VIEW -d "swiftly://debug_hidden" com.example.swiftly

    3. The screen is protected by a security check (`SecurityManager.check()`).
    4. Bypass this check using a `Frida hook` to allow access to the screen and view the `flag8Raw` field.

bypass.js:

```
Java.perform(function () {
    var SecurityManager = Java.use("com.example.swiftly.utils.security.SecurityManager");
    SecurityManager.check.implementation = function (ctx) {
        console.log("[+] SecurityManager.check() → forced true");
        return true;
    };
});
```

Command: frida -U -f com.example.swiftly -l bypass.js


Debug Console
Environment: SECURE
Security Status
Root detected
PASS
Debugger attached
PASS
App debuggable
WARN
Signature tampered
PASS
BUILD INFO
Package
com.example.swiftly
Version
1.0.0 (1)
Build
debug
API target
36
session
ZmxhZ19wYXJO
schema_ver
0x2A
flag_slot
UGF5bWVudHtkZWJ1Z19zY3JlZW5fZmxhZ184fQ==